

MedBooking

Проект реляционной базы данных и SQL

Предметная область: запись к врачу: пациенты, врачи, специальности, расписание, приемы.

Команда: Ергучев Александр Дмитриевич, Грюмова Кира Алексеевна, Тагиров Ранель Насимович.

Курс: Конструирование баз данных

Состав проекта

Файл	Назначение
sql/01_schema.sql	DDL: таблицы, ключи, ограничения, индексы, триггеры, представление свободных слотов
sql/02_seed.sql	Тестовое наполнение справочников, врачей, пациентов, расписания и приемов
sql/03_queries.sql	DML и аналитические запросы
sql/04_transactions.sql	Транзакции записи, переноса, отмены и завершения приема
diagrams/*.svg	Визуальные схемы для PDF
docker-compose.yml	PostgreSQL 16 через Docker Compose
.env.example	Пример переменных окружения для локального запуска
docs/ MedBooking_Project_Report.pdf	Итоговый PDF-документ

1. Пояснительная записка

1.1. Описание предметной области

Система MedBooking предназначена для поликлиники или медицинского центра, где пациенты записываются на прием к врачам определенных специальностей. Система хранит сведения о пациентах, врачах, специальностях, рабочих периодах врачей, временной недоступности врачей и фактических приемах.

Основные пользователи:

- регистратор создает карточки пациентов, ищет свободные слоты, записывает и отменяет приемы;
- врач видит свое расписание на день и историю приемов;
- администратор клиники управляет врачами, специальностями, рабочими периодами и анализирует загрузку;
- пациент выбирает специальность, врача, дату и свободное время приема.

Ключевое требование предметной области: два пациента не должны попасть к одному врачу на пересекающееся время. В проекте это ограничение реализовано в базе данных через `EXCLUDE USING gist`, поэтому оно защищает систему даже при конкурентных транзакциях.

1.2. Функциональные требования

Система должна поддерживать ведение справочника специальностей, регистрацию пациентов, ведение врачей и их специальностей, создание рабочих периодов, фиксацию недоступности врача, поиск свободных слотов, создание, перенос, подтверждение, завершение и отмену приема, просмотр расписания врача, историю пациента и аналитические отчеты.

Операции над данными: создание пациента, врача, расписания и приема; чтение свободных слотов, расписания и истории; обновление контактов, времени и статусов; физическое удаление только тестовых данных без зависимостей.

1.3. Нефункциональные требования

- **Согласованность:** запрет двойной записи находится на уровне БД.
- **Многопользовательский доступ:** конкурентные вставки в один слот завершаются одним успешным `COMMIT` и одной ошибкой ограничения.
- **Производительность:** добавлены индексы по врачу, пациенту, статусу и времени.

- Аудит: изменения статуса приема сохраняются в appointment_status_history.
- Сопровождаемость: ограничения выражены декларативно там, где это возможно.

1.4. Предварительная и итоговая схема

Сырая схема могла бы хранить все в одной таблице raw_appointments: данные пациента, врача, специальности, кабинета и приема. Такая схема быстро приводит к повторам и аномалиям. Итоговая модель разделяет независимые сущности.

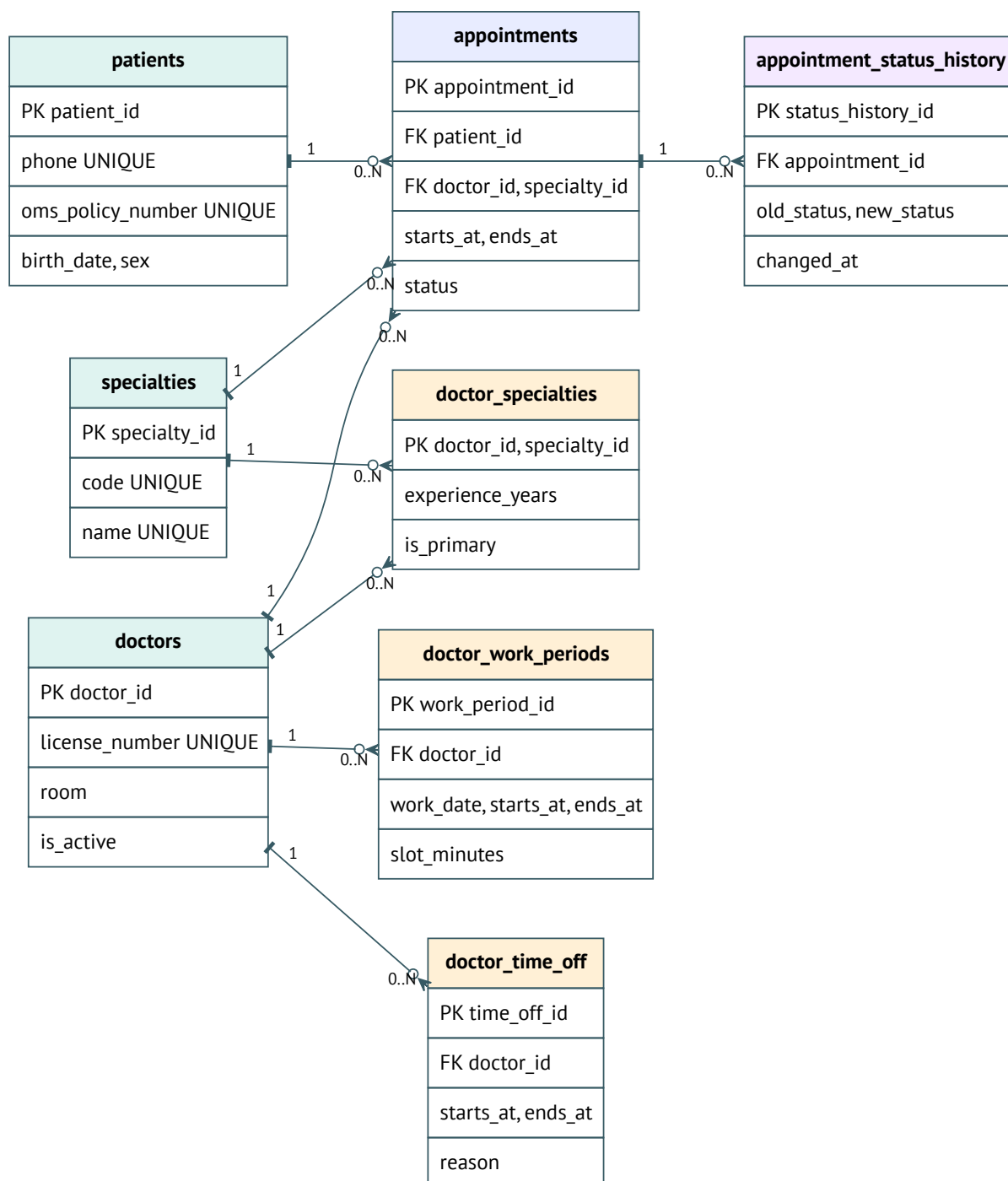


Рис. 1. ER-схема итоговой базы данных

1.5. Текстовые ограничения на данные

- Каждый пациент имеет уникальные телефон и номер полиса ОМС.
- Каждый врач имеет уникальную лицензию.

- Врач может иметь несколько специальностей.
- Прием относится к одному пациенту, врачу и специальности.
- Прием можно создать только по специальности, назначенной врачу.
- Активные приемы врача не пересекаются.
- Активные приемы пациента не пересекаются.
- Рабочие периоды врача не пересекаются.
- Прием создается только внутри рабочего периода и не во время недоступности врача.
- Отмененный прием обязан иметь дату и причину отмены.

1.6. Формализация ограничений

Функциональные зависимости:

- patient_id -> last_name, first_name, birth_date, sex, phone, email, oms_policy_number
- phone -> patient_id
- oms_policy_number -> patient_id
- doctor_id -> last_name, first_name, license_number, room, hire_date, is_active
- license_number -> doctor_id
- specialty_id -> code, name, description
- code -> specialty_id
- (doctor_id, specialty_id) -> experience_years, is_primary
- appointment_id -> patient_id, doctor_id, specialty_id, starts_at, ends_at, status, reason

Многозначные зависимости: doctor_id ->> specialty_id, doctor_id ->> work_period_id, patient_id ->> appointment_id, appointment_id ->> status_history_id.

1.7. Нормализация



Рис. 2. Переход от сырой таблицы к нормализованной схеме

1НФ устраняет повторяющиеся группы и неатомарные поля. 2НФ устраняет частичные зависимости в связующих таблицах. 3НФ выносит данные врача, пациента и специальности из приема. BCNF соблюдается для основных справочников и связей: детерминанты являются ключами или кандидатными ключами.

1.8. Анализ недонормализованной схемы

В таблице raw_appointments(patient_name, patient_phone, doctor_name, doctor_license, specialty_name, room, appointment_start, appointment_end) возникают:

- аномалия обновления: кабинет врача нужно менять во всех строках;
- аномалия вставки: нельзя добавить врача без приема;
- аномалия удаления: удаление последнего приема пациента удаляет сведения о пациенте.

Нормализация решает проблему, потому что пациент, врач, специальность и прием становятся самостоятельными сущностями.

1.9. SQL DDL

DDL вынесен в sql/01_schema.sql. Главные ограничения: исключаящие ограничения EXCLUDE USING gist для интервалов врача и пациента, составной FK (doctor_id, specialty_id), триггер проверки расписания, триггер аудита статусов.

1.10. SQL DML

Файл sql/03_queries.sql покрывает CRUD и сложные запросы: поиск свободных слотов, расписание врача, историю пациента, загрузку врачей, спрос по специальностям и рейтинг неявок.

1.11. Транзакции

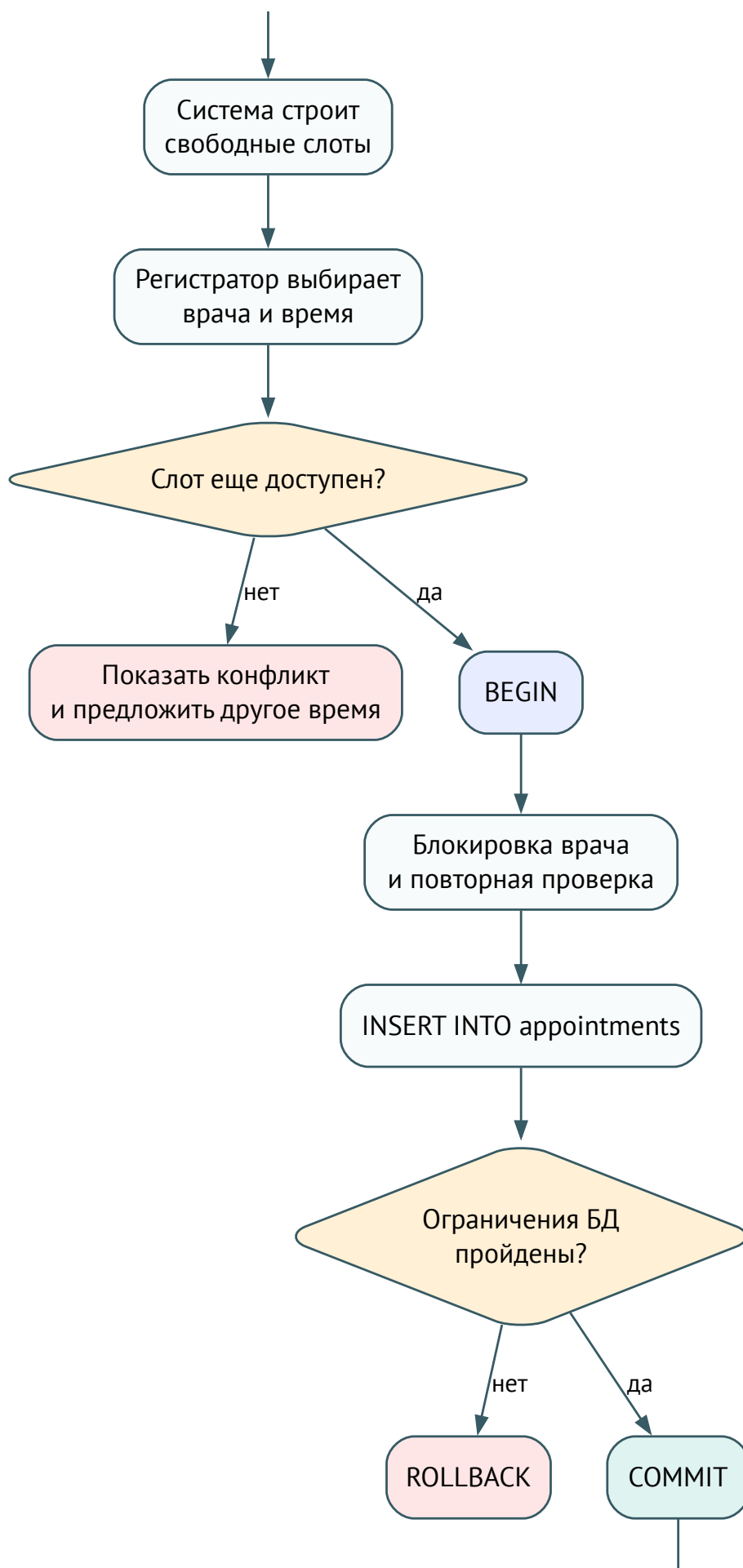


Рис. 3. Транзакционный сценарий записи на прием
стр. 6

Критичные транзакции: запись, перенос, отмена приемов при болезни врача, завершение приема. Границы транзакций выбраны так, чтобы состояние расписания и приемов менялось атомарно.

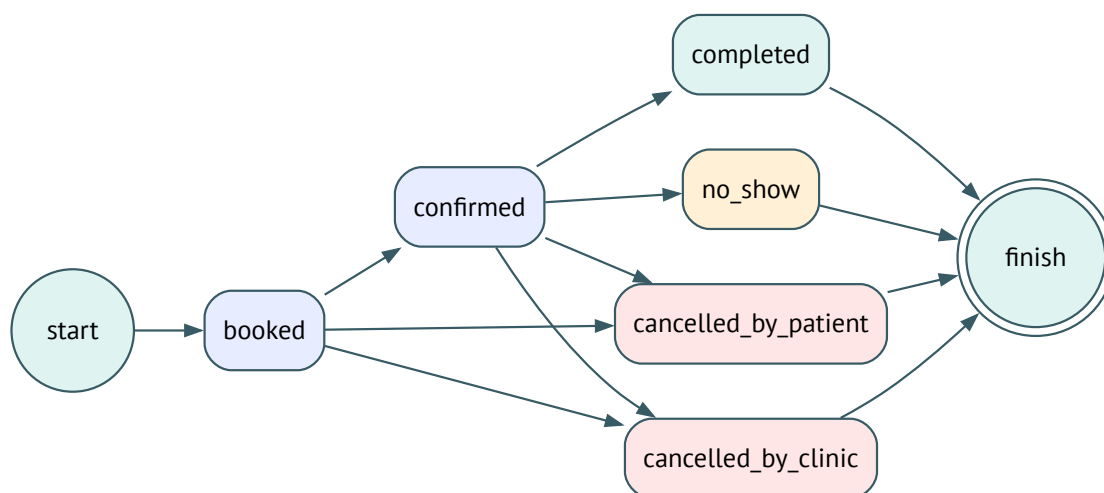


Рис. 4. Жизненный цикл статусов приема

1.12. Пользовательский интерфейс

Интерфейс не реализуется. Возможный минимальный UI: форма поиска слотов, карточка пациента, дневное расписание врача, административная панель.

2. Приложение по использованию AI

Раздел подготовлен как репрезентативная версия для итогового документа. Если преподаватель требует фактический журнал, замените фрагменты ниже реальными сообщениями команды и результатом реальной проверки второй моделью.

2.1. Репрезентативные фрагменты работы с основной AI-моделью

Фрагмент 1. Извлечение требований. Запрос: выделить сущности и ограничения для записи к врачу. Ответ: нужны пациенты, врачи, специальности, связь врача со специальностями, расписание и приемы; критичны запрет двойной записи и проверка рабочего времени. Уточнение: показать, какие ограничения должны быть реализованы именно в БД. Комментарий: полезно выделение пересечений интервалов; первоначально не хватало проверки специальности врача.

Фрагмент 2. Защита от двойной записи. Запрос: как надежно запретить запись двух пациентов к врачу на одно время в PostgreSQL. Ответ: уникальный индекс по (doctor_id, starts_at) слабее exclusion constraint по tsrange. Уточнение: добавить запрет пересечения активных приемов у пациента. Комментарий: итоговое решение устойчиво к разной длительности приемов.

Фрагмент 3. Нормализация. Запрос: почему нельзя хранить пациента, врача, специальность и прием в одной таблице. Ответ: возникают аномалии обновления, вставки и удаления. Уточнение: связать с 1НФ, 2НФ, 3НФ и BCNF. Комментарий: уточнение сделало раздел нормализации доказательным.

Фрагмент 4. Проверка DML. Запрос: покрывают ли запросы CRUD и сложную аналитику. Ответ: CRUD покрыт, сложные запросы используют JOIN, GROUP BY, агрегаты и оконные функции. Уточнение: добавить пользователя и ценность каждого запроса. Комментарий: запросы стали соответствовать формату задания.

2.2. Независимая верификация второй моделью

Каркас запроса:

Ты – старший архитектор баз данных с критическим стилем рецензирования. Я даю описание предметной области, логическую модель, DDL и SQL-запросы. Найди не менее 3 потенциальных проблем, спорных решений или слабых мест. Обрати внимание на аномалии, зависимости, нормализацию, ограничения целостности, производительность и альтернативы моделирования.

Репрезентативные замечания:

1. Уникального индекса по (doctor_id, starts_at) недостаточно при разной длительности приемов; нужен exclusion constraint.
2. Если прием хранит specialty_id, требуется проверка, что врач имеет эту специальность; нужен составной FK.
3. Проверка свободного времени только в приложении опасна; ограничения должны быть в БД.
4. Периоды недоступности врача не должны накладываться на активные приемы.
5. Для отчетов нужны индексы по врачу, пациенту и времени.

2.3. Комментарий команды по AI

После проверки добавлены исключающие ограничения для врача и пациента, составной FK для специальности врача, триггер проверки рабочего периода, представление свободных слотов и аудит статусов. Отклонено предложение хранить каждый слот отдельной строкой: в текущем проекте слоты вычисляются из рабочих периодов, что уменьшает объем данных.

Приложение A. DDL

```
BEGIN;

CREATE EXTENSION IF NOT EXISTS btree_gist;

DROP SCHEMA IF EXISTS medbooking CASCADE;
CREATE SCHEMA medbooking;
SET search_path = medbooking, public;

CREATE TYPE sex_code AS ENUM ('female', 'male');

CREATE TYPE appointment_status AS ENUM (
    'booked',
    'confirmed',
    'completed',
    'cancelled_by_patient',
    'cancelled_by_clinic',
    'no_show'
);

CREATE TABLE specialties (
    specialty_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    code text NOT NULL UNIQUE,
    name text NOT NULL UNIQUE,
    description text,
    CONSTRAINT specialties_code_format_chk
        CHECK (code ~ '^[a-z][a-z0-9_]{2,31}$'),
    CONSTRAINT specialties_name_not_blank_chk
        CHECK (length(trim(name)) >= 3)
);

CREATE TABLE patients (
    patient_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```



```

last_name text NOT NULL,
first_name text NOT NULL,
middle_name text,
birth_date date NOT NULL,
sex sex_code NOT NULL,
phone text NOT NULL,
email text,
oms_policy_number text NOT NULL,
created_at timestampz NOT NULL DEFAULT now(),
CONSTRAINT patients_last_name_not_blank_chk
    CHECK (length(trim(last_name)) >= 2),
CONSTRAINT patients_first_name_not_blank_chk
    CHECK (length(trim(first_name)) >= 2),
CONSTRAINT patients_birth_date_chk
    CHECK (birth_date < CURRENT_DATE),
CONSTRAINT patients_phone_format_chk
    CHECK (phone ~ '^\\+?[0-9]{10,15}$'),
CONSTRAINT patients_email_format_chk
    CHECK (email IS NULL OR email ~* '^\\[A-Z0-9._%+-]+@[A-Z0-9.-]+\\.\\[A-Z]{2,}$'),
CONSTRAINT patients_phone_uq UNIQUE (phone),
CONSTRAINT patients_oms_policy_uq UNIQUE (oms_policy_number)
);

CREATE UNIQUE INDEX patients_email_lower_uq
ON patients (lower(email))
WHERE email IS NOT NULL;

CREATE TABLE doctors (
    doctor_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    last_name text NOT NULL,
    first_name text NOT NULL,
    middle_name text,
    license_number text NOT NULL,
    phone text,
    email text,
    room text NOT NULL,
    hire_date date NOT NULL DEFAULT CURRENT_DATE,
    is_active boolean NOT NULL DEFAULT true,
    created_at timestampz NOT NULL DEFAULT now(),
CONSTRAINT doctors_last_name_not_blank_chk
    CHECK (length(trim(last_name)) >= 2),
CONSTRAINT doctors_first_name_not_blank_chk
    CHECK (length(trim(first_name)) >= 2),
CONSTRAINT doctors_license_not_blank_chk
    CHECK (length(trim(license_number)) >= 4),
CONSTRAINT doctors_room_not_blank_chk
    CHECK (length(trim(room)) >= 1),
CONSTRAINT doctors_phone_format_chk
    CHECK (phone IS NULL OR phone ~ '^\\+?[0-9]{10,15}$'),
CONSTRAINT doctors_email_format_chk
    CHECK (email IS NULL OR email ~* '^\\[A-Z0-9._%+-]+@[A-Z0-9.-]+\\.\\[A-Z]{2,}$'),
CONSTRAINT doctors_license_uq UNIQUE (license_number),
CONSTRAINT doctors_phone_uq UNIQUE (phone)
);

CREATE UNIQUE INDEX doctors_email_lower_uq
ON doctors (lower(email))
WHERE email IS NOT NULL;

```

```

CREATE TABLE doctor_specialties (
    doctor_id bigint NOT NULL REFERENCES doctors(doctor_id) ON DELETE CASCADE,
    specialty_id bigint NOT NULL REFERENCES specialties(specialty_id) ON DELETE RESTRICT,
    experience_years smallint NOT NULL DEFAULT 0,
    is_primary boolean NOT NULL DEFAULT false,
    PRIMARY KEY (doctor_id, specialty_id),
    CONSTRAINT doctor_specialties_experience_chk
        CHECK (experience_years BETWEEN 0 AND 70)
);

CREATE UNIQUE INDEX doctor_specialties_one_primary_uq
    ON doctor_specialties (doctor_id)
    WHERE is_primary;

CREATE INDEX doctor_specialties_specialty_idx
    ON doctor_specialties (specialty_id, doctor_id);

CREATE TABLE doctor_work_periods (
    work_period_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    doctor_id bigint NOT NULL REFERENCES doctors(doctor_id) ON DELETE CASCADE,
    work_date date NOT NULL,
    starts_at time NOT NULL,
    ends_at time NOT NULL,
    slot_minutes smallint NOT NULL DEFAULT 30,
    room text,
    created_at timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT doctor_work_periods_time_order_chk
        CHECK (starts_at < ends_at),
    CONSTRAINT doctor_work_periods_slot_minutes_chk
        CHECK (slot_minutes IN (10, 15, 20, 30, 45, 60)),
    CONSTRAINT doctor_work_periods_room_not_blank_chk
        CHECK (room IS NULL OR length(trim(room)) >= 1)
);

ALTER TABLE doctor_work_periods
    ADD CONSTRAINT doctor_work_periods_no_overlap
    EXCLUDE USING gist (
        doctor_id WITH =,
        tsrange(work_date + starts_at, work_date + ends_at, '[)') WITH &&
    );

CREATE INDEX doctor_work_periods_lookup_idx
    ON doctor_work_periods (doctor_id, work_date, starts_at);

CREATE TABLE doctor_time_off (
    time_off_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    doctor_id bigint NOT NULL REFERENCES doctors(doctor_id) ON DELETE CASCADE,
    starts_at timestamp without time zone NOT NULL,
    ends_at timestamp without time zone NOT NULL,
    reason text NOT NULL,
    created_at timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT doctor_time_off_time_order_chk
        CHECK (starts_at < ends_at),
    CONSTRAINT doctor_time_off_reason_not_blank_chk
        CHECK (length(trim(reason)) >= 3)
);

ALTER TABLE doctor_time_off
    ADD CONSTRAINT doctor_time_off_no_overlap

```

```

EXCLUDE USING gist (
    doctor_id WITH =,
    tsrange(starts_at, ends_at, '[') WITH &&
);

CREATE INDEX doctor_time_off_lookup_idx
ON doctor_time_off (doctor_id, starts_at, ends_at);

CREATE TABLE appointments (
    appointment_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    patient_id bigint NOT NULL REFERENCES patients(patient_id) ON DELETE RESTRICT,
    doctor_id bigint NOT NULL REFERENCES doctors(doctor_id) ON DELETE RESTRICT,
    specialty_id bigint NOT NULL REFERENCES specialties(specialty_id) ON DELETE RESTRICT,
    starts_at timestamp without time zone NOT NULL,
    ends_at timestamp without time zone NOT NULL,
    status appointment_status NOT NULL DEFAULT 'booked',
    reason text NOT NULL,
    created_at timestamptz NOT NULL DEFAULT now(),
    updated_at timestamptz NOT NULL DEFAULT now(),
    cancelled_at timestamptz,
    cancellation_reason text,
    created_by text NOT NULL DEFAULT current_user,
    CONSTRAINT appointments_time_order_chk
        CHECK (starts_at < ends_at),
    CONSTRAINT appointments_duration_min_chk
        CHECK (ends_at >= starts_at + interval '10 minutes'),
    CONSTRAINT appointments_duration_max_chk
        CHECK (ends_at <= starts_at + interval '2 hours'),
    CONSTRAINT appointments_reason_not_blank_chk
        CHECK (length(trim(reason)) >= 3),
    CONSTRAINT appointments_cancelled_state_chk
        CHECK (
            (
                status IN ('cancelled_by_patient', 'cancelled_by_clinic')
                AND cancelled_at IS NOT NULL
                AND cancellation_reason IS NOT NULL
                AND length(trim(cancellation_reason)) >= 3
            )
            OR
            (
                status NOT IN ('cancelled_by_patient', 'cancelled_by_clinic')
                AND cancelled_at IS NULL
                AND cancellation_reason IS NULL
            )
        ),
    CONSTRAINT appointments_doctor_specialty_fk
        FOREIGN KEY (doctor_id, specialty_id)
        REFERENCES doctor_specialties(doctor_id, specialty_id)
        ON DELETE RESTRICT
);

ALTER TABLE appointments
ADD CONSTRAINT appointments_no_doctor_overlap
EXCLUDE USING gist (
    doctor_id WITH =,
    tsrange(starts_at, ends_at, '[') WITH &&
)
WHERE (status IN ('booked', 'confirmed', 'completed', 'no_show'));

```

```

ALTER TABLE appointments
  ADD CONSTRAINT appointments_no_patient_overlap
  EXCLUDE USING gist (
    patient_id WITH =,
    tsrange(starts_at, ends_at, '[') WITH &&
  )
  WHERE (status IN ('booked', 'confirmed', 'completed', 'no_show'));

CREATE INDEX appointments_doctor_starts_idx
  ON appointments (doctor_id, starts_at);

CREATE INDEX appointments_patient_starts_idx
  ON appointments (patient_id, starts_at);

CREATE INDEX appointments_specialty_status_idx
  ON appointments (specialty_id, status, starts_at);

CREATE INDEX appointments_active_idx
  ON appointments (starts_at, doctor_id, patient_id)
  WHERE status IN ('booked', 'confirmed');

CREATE TABLE appointment_status_history (
  status_history_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  appointment_id bigint NOT NULL REFERENCES appointments(appointment_id) ON DELETE
  CASCADE,
  old_status appointment_status,
  new_status appointment_status NOT NULL,
  changed_at timestamptz NOT NULL DEFAULT now(),
  actor text NOT NULL DEFAULT current_user,
  comment text
);

CREATE OR REPLACE FUNCTION set_updated_at()
  RETURNS trigger
  LANGUAGE plpgsql
  AS $$
  BEGIN
    NEW.updated_at := now();
    RETURN NEW;
  END;
  $$;

CREATE TRIGGER trg_appointments_set_updated_at
  BEFORE UPDATE ON appointments
  FOR EACH ROW
  EXECUTE FUNCTION set_updated_at();

CREATE OR REPLACE FUNCTION audit_appointment_status()
  RETURNS trigger
  LANGUAGE plpgsql
  AS $$
  BEGIN
    IF TG_OP = 'INSERT' THEN
      INSERT INTO appointment_status_history (
        appointment_id,
        old_status,
        new_status,
        comment
      )
    
```

```

VALUES (
    NEW.appointment_id,
    NULL,
    NEW.status,
    'начальный статус'
);
RETURN NEW;
END IF;

IF OLD.status IS DISTINCT FROM NEW.status THEN
    INSERT INTO appointment_status_history (
        appointment_id,
        old_status,
        new_status,
        comment
    )
    VALUES (
        NEW.appointment_id,
        OLD.status,
        NEW.status,
        'статус изменен'
    );
END IF;

RETURN NEW;
END;
$$;

CREATE TRIGGER trg_appointments_audit_status
AFTER INSERT OR UPDATE OF status ON appointments
FOR EACH ROW
EXECUTE FUNCTION audit_appointment_status();

CREATE OR REPLACE FUNCTION validate_appointment_schedule()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.status IN ('booked', 'confirmed') THEN
        IF NOT EXISTS (
            SELECT 1
            FROM doctor_work_periods wp
            JOIN doctors d ON d.doctor_id = wp.doctor_id
            WHERE wp.doctor_id = NEW.doctor_id
            AND d.is_active
            AND wp.work_date = NEW.starts_at::date
            AND NEW.ends_at::date = NEW.starts_at::date
            AND NEW.starts_at::time >= wp.starts_at
            AND NEW.ends_at::time <= wp.ends_at
            AND EXTRACT(EPOCH FROM (NEW.ends_at - NEW.starts_at)) / 60 = wp.slot_minutes
            AND mod(
                (EXTRACT(EPOCH FROM (NEW.starts_at::time - wp.starts_at)) / 60)::int,
                wp.slot_minutes
            ) = 0
        ) THEN
            RAISE EXCEPTION
            'Прием % находится вне рабочего периода врача или сетки слотов',
            NEW.appointment_id
            USING ERRCODE = '23514';
        END IF;
    END IF;
END;

```

```

        END IF;

        IF EXISTS (
            SELECT 1
            FROM doctor_time_off t
            WHERE t.doctor_id = NEW.doctor_id
            AND tsrange(t.starts_at, t.ends_at, '[')
            && tsrange(NEW.starts_at, NEW.ends_at, '[')
        ) THEN
            RAISE EXCEPTION
                'Врач % недоступен в выбранный интервал',
                NEW.doctor_id
            USING ERRCODE = '23514';
        END IF;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validate_appointment_schedule
BEFORE INSERT OR UPDATE OF doctor_id, starts_at, ends_at, status ON appointments
FOR EACH ROW
EXECUTE FUNCTION validate_appointment_schedule();

CREATE OR REPLACE FUNCTION validate_time_off_has_no_active_appointments()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM appointments a
        WHERE a.doctor_id = NEW.doctor_id
        AND a.status IN ('booked', 'confirmed')
        AND tsrange(a.starts_at, a.ends_at, '[')
        && tsrange(NEW.starts_at, NEW.ends_at, '[')
    ) THEN
        RAISE EXCEPTION
            'Нельзя создать период отсутствия врача поверх активных приемов'
        USING ERRCODE = '23514';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validate_time_off_has_no_active_appointments
BEFORE INSERT OR UPDATE OF doctor_id, starts_at, ends_at ON doctor_time_off
FOR EACH ROW
EXECUTE FUNCTION validate_time_off_has_no_active_appointments();

CREATE VIEW v_doctor_work_slots AS
SELECT
    wp.doctor_id,
    d.last_name || ' ' || d.first_name AS doctor_name,
    ds.specialty_id,
    s.name AS specialty_name,
    wp.work_date,

```

```

        gs.slot_start,
        gs.slot_start + make_interval(mins => wp.slot_minutes) AS slot_end,
        COALESCE(wp.room, d.room) AS room
FROM doctor_work_periods wp
JOIN doctors d ON d.doctor_id = wp.doctor_id
JOIN doctor_specialties ds ON ds.doctor_id = d.doctor_id
JOIN specialties s ON s.specialty_id = ds.specialty_id
CROSS JOIN LATERAL generate_series(
    wp.work_date + wp.starts_at,
    wp.work_date + wp.ends_at - make_interval(mins => wp.slot_minutes),
    make_interval(mins => wp.slot_minutes)
) AS gs(slot_start)
WHERE d.is_active
    AND NOT EXISTS (
        SELECT 1
        FROM appointments a
        WHERE a.doctor_id = wp.doctor_id
            AND a.status IN ('booked', 'confirmed')
            AND tsrange(a.starts_at, a.ends_at, '[]')
                && tsrange(gs.slot_start, gs.slot_start + make_interval(mins =>
wp.slot_minutes), '[]')
    )
    AND NOT EXISTS (
        SELECT 1
        FROM doctor_time_off t
        WHERE t.doctor_id = wp.doctor_id
            AND tsrange(t.starts_at, t.ends_at, '[]')
                && tsrange(gs.slot_start, gs.slot_start + make_interval(mins =>
wp.slot_minutes), '[]')
    );

COMMIT;

```

Приложение В. Тестовые данные

```

BEGIN;

SET search_path = medbooking, public;

TRUNCATE TABLE
    appointment_status_history,
    appointments,
    doctor_time_off,
    doctor_work_periods,
    doctor_specialties,
    doctors,
    patients,
    specialties
RESTART IDENTITY CASCADE;

INSERT INTO specialties (code, name, description) VALUES
    ('therapy', 'Терапия', 'Первичный прием взрослых пациентов'),
    ('cardiology', 'Кардиология', 'Диагностика и лечение сердечно-сосудистых заболеваний'),
    ('neurology', 'Неврология', 'Диагностика заболеваний нервной системы'),
    ('pediatrics', 'Педиатрия', 'Прием детей и подростков'),
    ('dermatology', 'Дерматология', 'Заболевания кожи');

INSERT INTO doctors (
    last_name,

```

```

first_name,
middle_name,
license_number,
phone,
email,
room,
hire_date
) VALUES
  ('Соколова', 'Анна', 'Петровна', 'LIC-THER-001', '+79001001001',
'sokolova@medbooking.local', '201', DATE '2020-02-10'),
  ('Орлов', 'Дмитрий', 'Игоревич', 'LIC-CARD-002', '+79001001002',
'orlov@medbooking.local', '305', DATE '2018-09-03'),
  ('Лебедева', 'Мария', 'Сергеевна', 'LIC-NEUR-003', '+79001001003',
'lebedeva@medbooking.local', '214', DATE '2021-01-15'),
  ('Кузнецов', 'Алексей', 'Викторович', 'LIC-PED-004', '+79001001004',
'kuznetsov@medbooking.local', '118', DATE '2019-06-20');

INSERT INTO doctor_specialties (doctor_id, specialty_id, experience_years, is_primary)
VALUES
  (1, 1, 11, true),
  (1, 5, 4, false),
  (2, 2, 15, true),
  (3, 3, 9, true),
  (4, 4, 13, true),
  (4, 1, 7, false);

INSERT INTO patients (
  last_name,
  first_name,
  middle_name,
  birth_date,
  sex,
  phone,
  email,
  oms_policy_number
) VALUES
  ('Иванов', 'Павел', 'Олегович', DATE '1988-04-12', 'male', '+79110000001',
'ivanov@example.local', '7700000000000001'),
  ('Петрова', 'Елена', 'Андреевна', DATE '1994-11-03', 'female', '+79110000002',
'petrova@example.local', '7700000000000002'),
  ('Смирнов', 'Илья', 'Максимович', DATE '1979-01-25', 'male', '+79110000003',
'smirnov@example.local', '7700000000000003'),
  ('Морозова', 'Алина', 'Романовна', DATE '2015-07-09', 'female', '+79110000004',
'morozova@example.local', '7700000000000004'),
  ('Волкова', 'Наталья', 'Ильинична', DATE '1968-02-18', 'female', '+79110000005',
'volkova@example.local', '7700000000000005');

INSERT INTO doctor_work_periods (
  doctor_id,
  work_date,
  starts_at,
  ends_at,
  slot_minutes,
  room
) VALUES
  (1, DATE '2026-06-15', TIME '09:00', TIME '15:00', 30, '201'),
  (1, DATE '2026-06-16', TIME '09:00', TIME '15:00', 30, '201'),
  (2, DATE '2026-06-15', TIME '10:00', TIME '16:00', 30, '305'),
  (2, DATE '2026-06-16', TIME '10:00', TIME '16:00', 30, '305'),

```



```

(3, DATE '2026-06-15', TIME '09:00', TIME '13:00', 30, '214'),
(4, DATE '2026-06-15', TIME '08:30', TIME '14:30', 30, '118'),
(4, DATE '2026-06-16', TIME '08:30', TIME '14:30', 30, '118');

INSERT INTO doctor_time_off (
    doctor_id,
    starts_at,
    ends_at,
    reason
) VALUES
(1, TIMESTAMP '2026-06-15 13:00', TIMESTAMP '2026-06-15 14:00', 'Совещание отделения');

INSERT INTO appointments (
    patient_id,
    doctor_id,
    specialty_id,
    starts_at,
    ends_at,
    status,
    reason
) VALUES
(1, 1, 1, TIMESTAMP '2026-06-15 09:00', TIMESTAMP '2026-06-15 09:30', 'booked',
'Первичный прием'),
(2, 1, 1, TIMESTAMP '2026-06-15 09:30', TIMESTAMP '2026-06-15 10:00', 'confirmed',
'Контроль анализов'),
(3, 2, 2, TIMESTAMP '2026-06-15 10:00', TIMESTAMP '2026-06-15 10:30', 'booked', 'Боль в
груди'),
(4, 4, 4, TIMESTAMP '2026-06-15 08:30', TIMESTAMP '2026-06-15 09:00', 'confirmed',
'Профилактический осмотр'),
(5, 2, 2, TIMESTAMP '2026-06-16 10:00', TIMESTAMP '2026-06-16 10:30', 'booked',
'Плановая консультация');

INSERT INTO appointments (
    patient_id,
    doctor_id,
    specialty_id,
    starts_at,
    ends_at,
    status,
    reason,
    cancelled_at,
    cancellation_reason
) VALUES
(5, 3, 3, TIMESTAMP '2026-06-15 09:00', TIMESTAMP '2026-06-15 09:30',
'cancelled_by_patient', 'Головные боли', now(), 'Пациент выбрал другую дату');

COMMIT;

```

Приложение C. DML-запросы

```

SET search_path = medbooking, public;

INSERT INTO patients (
    last_name,
    first_name,
    middle_name,
    birth_date,
    sex,
    phone,

```

```

    email,
    oms_policy_number
) VALUES (
    'Никитина',
    'Ольга',
    'Павловна',
    DATE '1991-05-17',
    'female',
    '+791100000006',
    'nikitina@example.local',
    '7700000000000006'
);

SELECT
    slot_start,
    slot_end,
    doctor_id,
    doctor_name,
    room
FROM v_doctor_work_slots
WHERE specialty_id = (
    SELECT specialty_id
    FROM specialties
    WHERE code = 'therapy'
)
AND work_date = DATE '2026-06-15'
ORDER BY slot_start, doctor_name;

SELECT
    a.starts_at,
    a.ends_at,
    a.status,
    p.last_name || ' ' || p.first_name AS patient_name,
    p.phone,
    s.name AS specialty,
    a.reason
FROM appointments a
JOIN patients p ON p.patient_id = a.patient_id
JOIN specialties s ON s.specialty_id = a.specialty_id
WHERE a.doctor_id = 1
    AND a.starts_at::date = DATE '2026-06-15'
ORDER BY a.starts_at;

UPDATE patients
SET phone = '+79119990006',
    email = 'nikitina.new@example.local'
WHERE oms_policy_number = '7700000000000006';

UPDATE appointments
SET status = 'cancelled_by_patient',
    cancelled_at = now(),
    cancellation_reason = 'Пациент отказался от визита'
WHERE appointment_id = 1
    AND status IN ('booked', 'confirmed');

DELETE FROM patients p
WHERE p.oms_policy_number = '7700000000000006'
    AND NOT EXISTS (
        SELECT 1

```

```

        FROM appointments a
        WHERE a.patient_id = p.patient_id
    );

SELECT
    p.last_name || ' ' || p.first_name AS patient_name,
    a.starts_at,
    d.last_name || ' ' || d.first_name AS doctor_name,
    s.name AS specialty,
    a.status,
    row_number() OVER (
        PARTITION BY a.patient_id
        ORDER BY a.starts_at DESC
    ) AS visit_number_desc
FROM appointments a
JOIN patients p ON p.patient_id = a.patient_id
JOIN doctors d ON d.doctor_id = a.doctor_id
JOIN specialties s ON s.specialty_id = a.specialty_id
WHERE p.oms_policy_number = '7700000000000002'
ORDER BY a.starts_at DESC;

SELECT
    d.doctor_id,
    d.last_name || ' ' || d.first_name AS doctor_name,
    a.starts_at::date AS work_date,
    count(*) FILTER (WHERE a.status IN ('booked', 'confirmed', 'completed')) AS
active_or_done_count,
    count(*) FILTER (WHERE a.status IN ('cancelled_by_patient', 'cancelled_by_clinic')) AS
cancelled_count,
    round(
        100.0 * count(*) FILTER (WHERE a.status IN ('booked', 'confirmed', 'completed'))
        / NULLIF(count(*), 0),
        2
    ) AS useful_load_percent
FROM appointments a
JOIN doctors d ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, doctor_name, a.starts_at::date
ORDER BY work_date, useful_load_percent DESC;

SELECT
    s.name AS specialty,
    count(a.appointment_id) AS total_appointments,
    count(*) FILTER (WHERE a.status = 'completed') AS completed_count,
    count(*) FILTER (WHERE a.status IN ('booked', 'confirmed')) AS future_active_count,
    count(DISTINCT a.patient_id) AS unique_patients
FROM specialties s
LEFT JOIN appointments a ON a.specialty_id = s.specialty_id
GROUP BY s.specialty_id, s.name
ORDER BY total_appointments DESC, specialty;

WITH doctor_stats AS (
    SELECT
        d.doctor_id,
        d.last_name || ' ' || d.first_name AS doctor_name,
        count(*) AS total_appointments,
        count(*) FILTER (WHERE a.status = 'no_show') AS no_show_count
    FROM doctors d
    LEFT JOIN appointments a ON a.doctor_id = d.doctor_id
    GROUP BY d.doctor_id, doctor_name

```

```

)
SELECT
    doctor_name,
    total_appointments,
    no_show_count,
    round(100.0 * no_show_count / NULLIF(total_appointments, 0), 2) AS no_show_percent,
    rank() OVER (
        ORDER BY 1.0 * no_show_count / NULLIF(total_appointments, 0) DESC NULLS LAST
    ) AS risk_rank
FROM doctor_stats
ORDER BY risk_rank, doctor_name;

```

Приложение D. Транзакции

```

SET search_path = medbooking, public;

BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET LOCAL lock_timeout = '5s';

SELECT doctor_id
FROM doctors
WHERE doctor_id = 1
    AND is_active
FOR UPDATE;

SELECT slot_start, slot_end
FROM v_doctor_work_slots
WHERE doctor_id = 1
    AND specialty_id = 1
    AND slot_start = TIMESTAMP '2026-06-15 10:30';

INSERT INTO appointments (
    patient_id,
    doctor_id,
    specialty_id,
    starts_at,
    ends_at,
    status,
    reason
) VALUES (
    5,
    1,
    1,
    TIMESTAMP '2026-06-15 10:30',
    TIMESTAMP '2026-06-15 11:00',
    'booked',
    'Повторная консультация'
);

COMMIT;

BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET LOCAL lock_timeout = '5s';

SELECT appointment_id
FROM appointments
WHERE appointment_id = 3

```

```

    AND status IN ('booked', 'confirmed')
FOR UPDATE;

UPDATE appointments
SET starts_at = TIMESTAMP '2026-06-15 11:00',
    ends_at = TIMESTAMP '2026-06-15 11:30'
WHERE appointment_id = 3
    AND status IN ('booked', 'confirmed');

COMMIT;

BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET LOCAL lock_timeout = '5s';

SELECT doctor_id
FROM doctors
WHERE doctor_id = 2
FOR UPDATE;

UPDATE appointments
SET status = 'cancelled_by_clinic',
    cancelled_at = now(),
    cancellation_reason = 'Врач отсутствует по болезни'
WHERE doctor_id = 2
    AND status IN ('booked', 'confirmed')
    AND tsrange(starts_at, ends_at, '[')
        && tsrange(TIMESTAMP '2026-06-16 10:00', TIMESTAMP '2026-06-16 13:00', '['));

INSERT INTO doctor_time_off (
    doctor_id,
    starts_at,
    ends_at,
    reason
) VALUES (
    2,
    TIMESTAMP '2026-06-16 10:00',
    TIMESTAMP '2026-06-16 13:00',
    'Больничный'
);

COMMIT;

BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET LOCAL lock_timeout = '5s';

SELECT appointment_id
FROM appointments
WHERE appointment_id = 2
FOR UPDATE;

UPDATE appointments
SET status = 'completed'
WHERE appointment_id = 2
    AND status = 'confirmed';

COMMIT;

```